



Siete modelos de concurrency

Paul Butcher

Septiembre de 2026

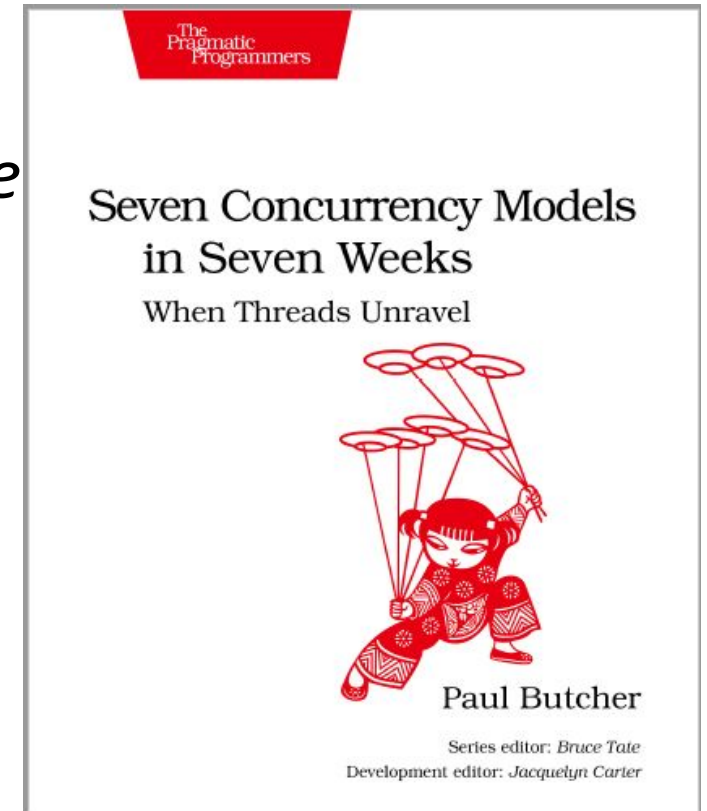
Seven Concurrency Models

• *Introduction*

1. *Threads and locks*
2. *Functional programming*
3. *The Clojure Way—separating identity and state*
4. *Actors*
5. *Communicating Sequential Processes*
6. *Data parallelism*
7. *The Lambda Architecture*
8. *Wrapping Up*

Vamos a dar una
mirada rápida a
este libro

2014



Introducción

- ¿Concurrente o Paralelo?
 - Un programa concurrente tiene múltiples hilos lógicos de control. Estos subprocesos pueden o no ejecutarse en paralelo.
 - Un programa paralelo potencialmente se ejecuta más rápidamente que un programa secuencial al ejecutar diferentes partes del cálculo simultáneamente (en paralelo). Puede o no tener más de un hilo lógico de control.
- Arquitectura paralela: los computadores son paralelos en diferentes niveles
 - Paralelismo a nivel de Bit (¿Por qué una computadora de 32 bits es más rápida que una de 8 bits?)
 - Paralelismo a nivel de instrucción (uso de pipelining, ejecución fuera de orden y ejecución especulativa, pero ojo con el “NO determinismo”)
 - Paralelismo de datos (SIMD, para “single instruction, multiple data”)
 - Paralelismo a nivel de tarea (múltiples procesadores, pero es importante el modelo de memoria, ¿*shared memory* o *distributed memory*?)

Introducción

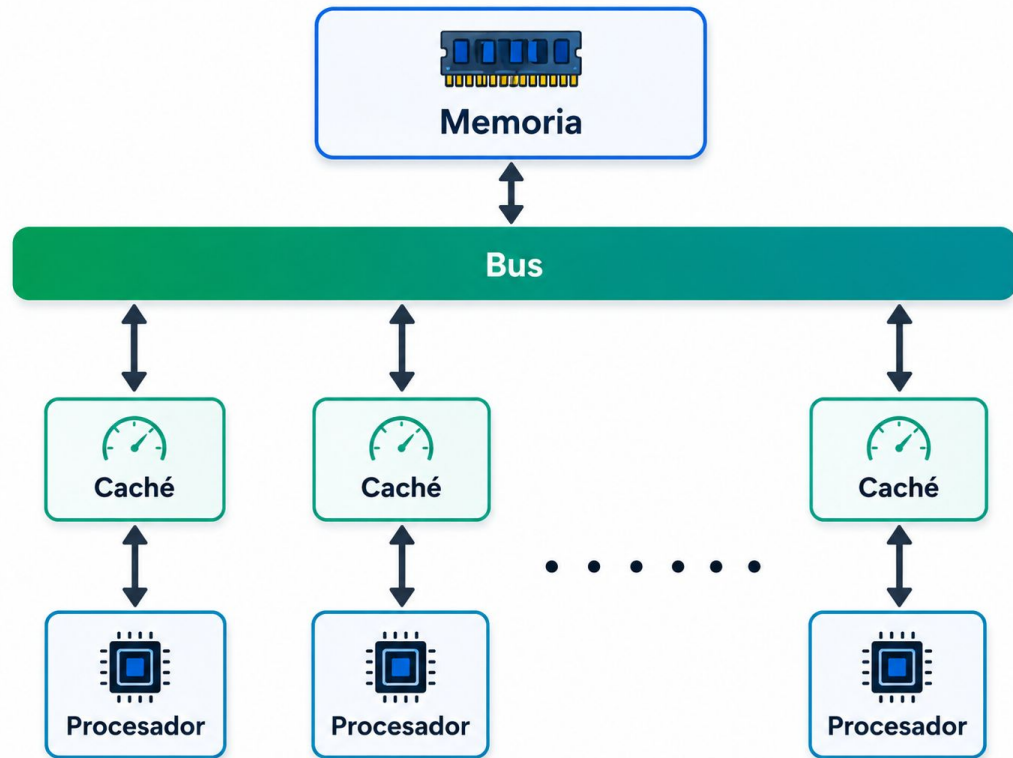


Figura 1 — Memoria compartida

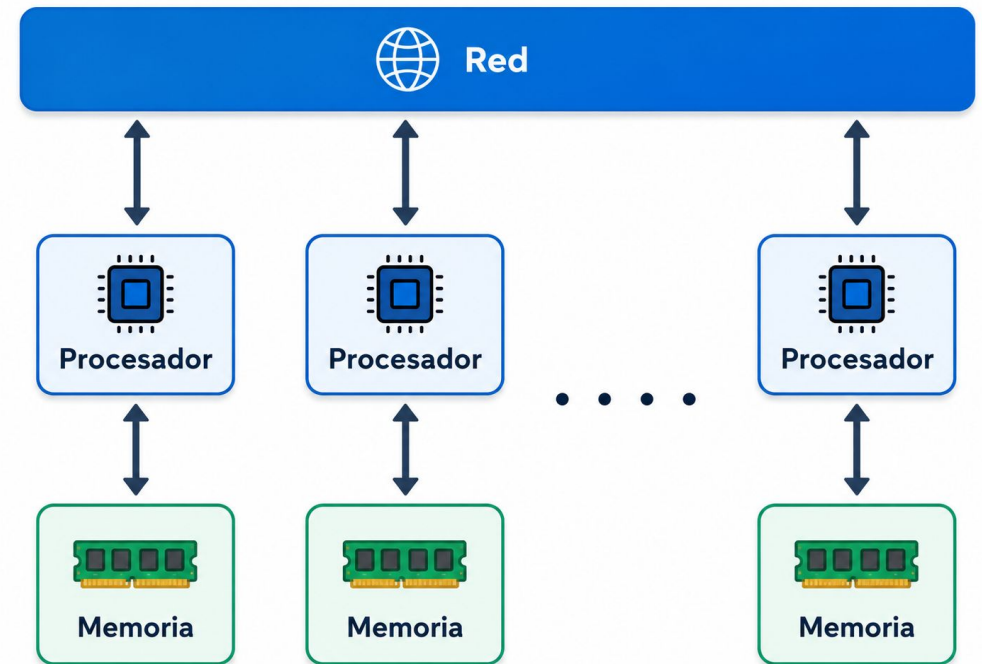


Figura 2 — Memoria distribuida

Introducción

- Software concurrente para un mundo concurrente
 - Su teléfono móvil puede reproducir música, hablar a la red y prestar atención a su dedo tocando su pantalla, todo al mismo tiempo.
 - La concurrencia es la clave para sistemas *responsive* [con capacidad de respuesta].
- Software distribuido para un mundo distribuido
 - A veces, la distribución geográfica es un elemento clave del problema que está resolviendo. La distribución de software ayuda a manejar las fallas.
- Software resiliente para un mundo impredecible
 - La concurrencia permite un software resiliente o tolerante a fallas a través de la independencia y la detección de fallas.
- Software simple en un mundo complejo
 - Una solución concurrente puede ser más simple y clara que su equivalente secuencial cuando se escribe en el lenguaje correcto con las herramientas adecuadas.

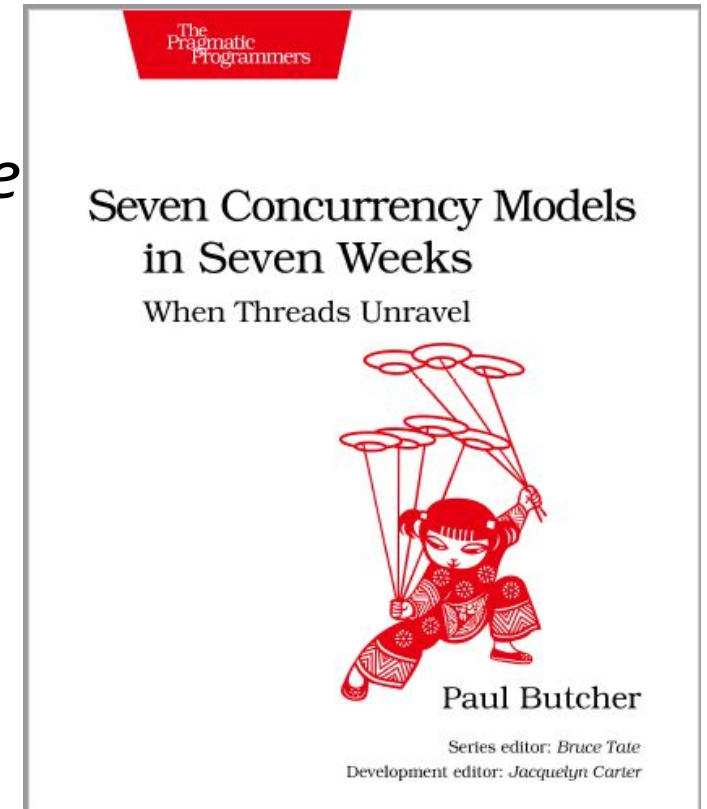
Seven Concurrency Models

- *Introduction*

1. *Threads and locks*
2. *Functional programming*
3. *The Clojure Way—separating identity and state*
4. *Actors*
5. *Communicating Sequential Processes*
6. *Data parallelism*
7. *The Lambda Architecture*
8. *Wrapping Up*

Dando una mirada
rápida a este libro

2014



Hilos y cerrojos (Threads and locks)

- La programación con hilos y cerrojos es como un vehículo Ford modelo T. Puede llevarnos del punto A al punto B, pero es primitivo, difícil de conducir, poco confiable –incluso peligroso- comparado con nuevas tecnologías.
- Olvidando un poco sus problemas, la programación con hilos y cerrojos aún se utilizan para escribir mucho software concurrente y como soporta otras tecnologías concurrentes, es bueno estudiarla.



Algo muy simple

- Los hilos y los cerrojos (Locks) son poco más que una formalización de lo que subyace el hardware realmente lo hace. Esa es su gran fuerza y, a la vez, su gran debilidad.
- Debido a que son tan simples, casi todos los lenguajes los admiten de una forma u otra, e imponen muy pocas restricciones sobre lo que se puede lograr a través de su uso.
- Pero tampoco proporcionan casi ninguna ayuda al pobre programador, haciendo que los programas sean muy difíciles de acertar en primer lugar e incluso más difícil de mantener.
- En el capítulo usan el lenguaje Java
 - Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java concurrency in practice*. Addison-Wesley Professional.
 - Urma, R.-G., Fusco, M., & Mycroft, A. (2018). *Modern Java in action: Lambdas, streams, functional and reactive programming*. Manning Publications.

Actividades propuestas para cada día

- Day 1: Mutual Exclusion and Memory Models
 - [Día 1: Exclusión mutua y modelos de memoria]
- Day 2: Beyond Intrinsic Locks
 - [Día 2: Más allá de los cerrojos intrínsecas]
- Day 3: On the Shoulders of Giants
 - [Día 3: Sobre los hombros de gigantes]



Conclusión del capítulo



• Fortalezas

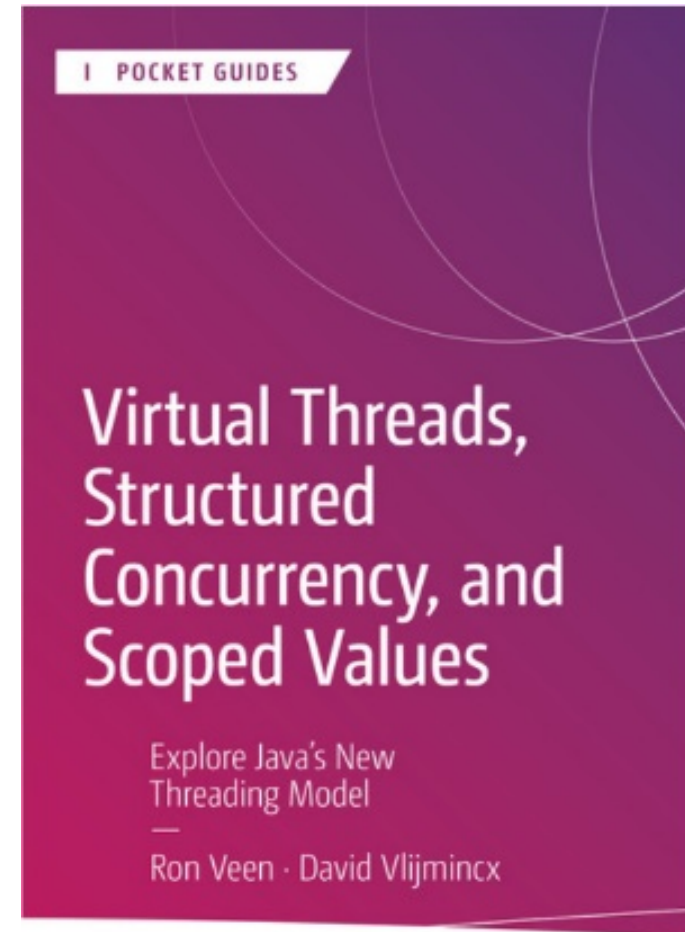
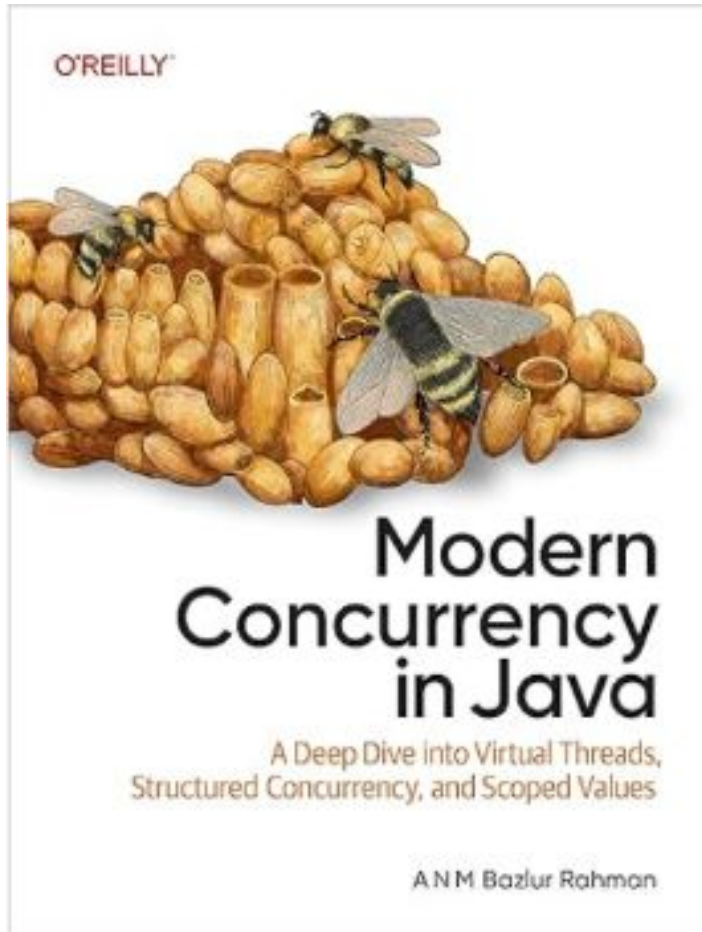
- Pueden aplicarse a una amplia gama de problemas.
- Pueden ser muy eficientes cuando se utilizan correctamente.
- Pueden integrarse fácilmente en la mayoría de los lenguajes de programación.



• Debilidades

- No proporcionan soporte directo para el paralelismo
- Se pueden utilizar para paralelizar un algoritmo secuencial pero introducen no determinismo.
- Sólo soportan arquitecturas de memoria compartida.
 - Si necesita soportar memoria distribuida (y, por extensión, distribución geográfica o resiliencia), tendrá que buscar en otra parte
- La programación de hilos y cerrojos es difícil [muy poca ayuda].
- Pruebas y mantenimiento son difíciles

Hilos virtuales: se estudiará de otros libros



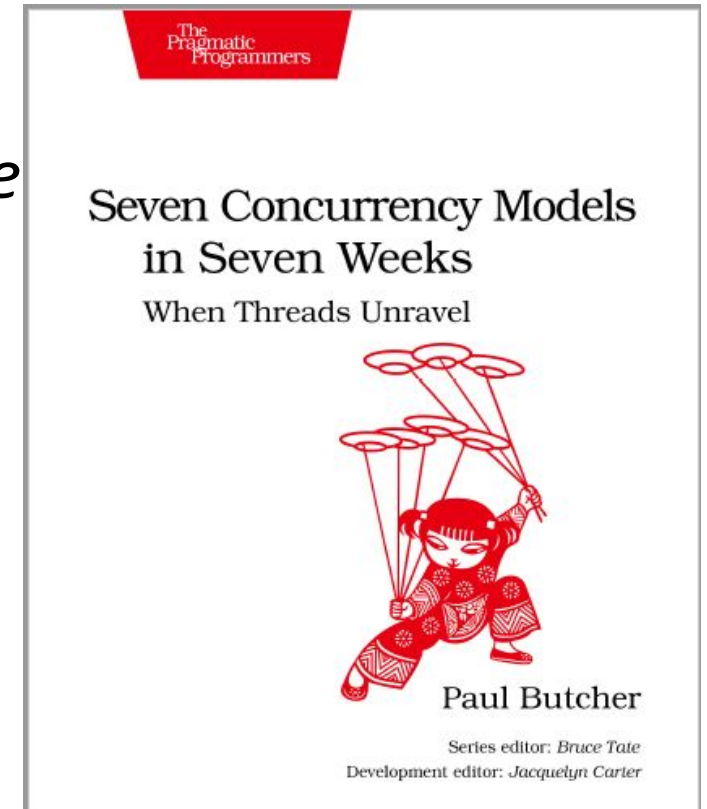
Seven Concurrency Models

- *Introduction*

1. *Threads and locks*
2. *Functional programming*
3. *The Clojure Way—separating identity and state*
4. *Actors*
5. *Communicating Sequential Processes*
6. *Data parallelism*
7. *The Lambda Architecture*
8. *Wrapping Up*

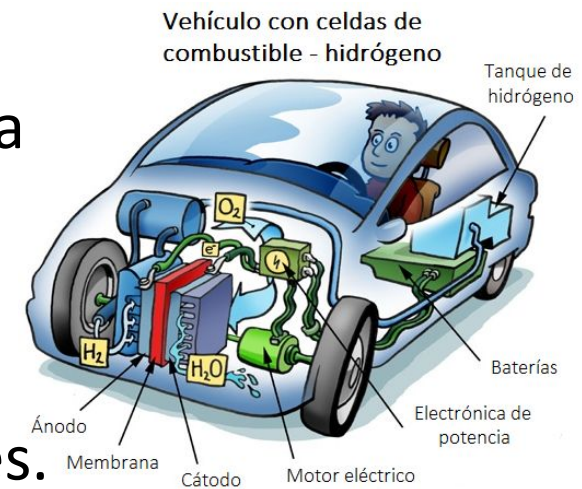
Dando una mirada
rápida a este libro

2014



Programación funcional (Functional programming)

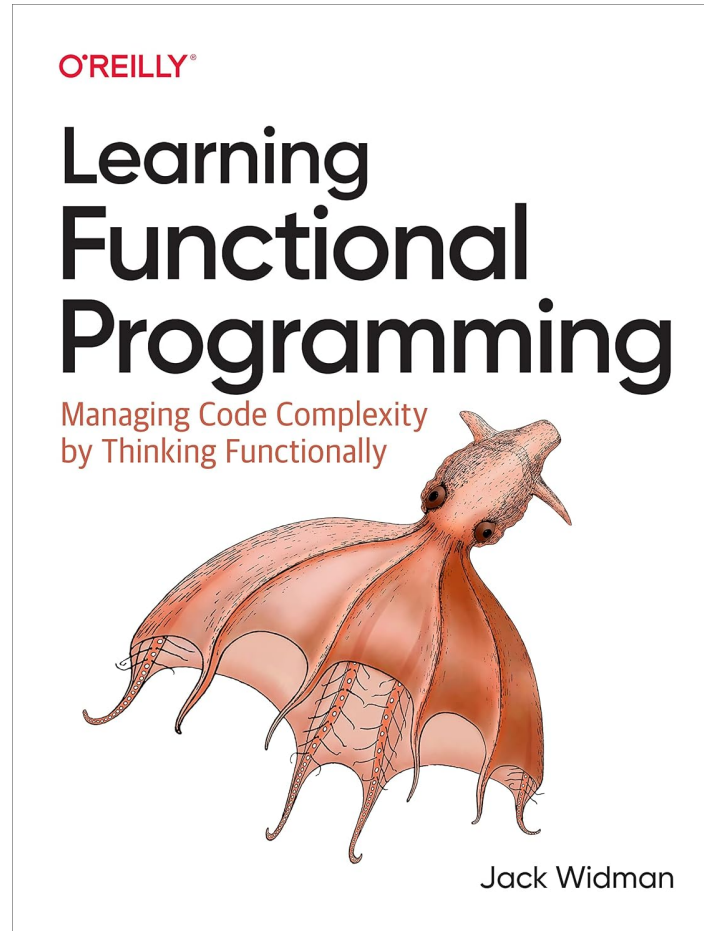
- La programación funcional es como un vehículo alimentado con celdas de combustible basado en hidrógeno: avanzado, futurista y aún no ampliamente utilizado, pero que será común en algunas décadas.
- En contraste con la programación imperativa, que consta de una serie de sentencias que cambian el estado global cuando es ejecutado, un modelo de programación funcional procesa haciendo evaluaciones de expresiones que son construidas a partir de funciones matemáticas puras: son de primera clase (pueden ser manipuladas como otro valor) y sin efectos laterales.
 - La ausencia de efectos laterales facilita el razonamiento acerca de los hilos, haciendo más seguro y más fácil
- También es el primer modelo presentado aquí que permite representar el paralelismo directamente.



Si duele, deje de hacerlo

- Los datos que no cambian (son *inmutables*) pueden ser accedidos por múltiples hilos sin ningún tipo de cerrojo.
 - Esto es lo que hace que la programación funcional sea tan convincente cuando se trata de concurrencia y paralelismo: los programas funcionales no tienen estado mutable, por lo que no pueden sufrir ninguno de los problemas asociados con el estado mutable compartido.
- En este capítulo [en este libro] utilizan Clojure (<http://clojure.org>)
 - Un dialecto de Lisp que se ejecuta en la JVM. Clojure está tipado dinámicamente; y si usted es programador de Ruby o Python, se sentirá como en casa una vez que te acostumbres a la sintaxis desconocida. Clojure no es un lenguaje funcional puro.
 - Compruebe antes si ya tiene Java instalado.
 - <https://applab.unc.edu/posts/2019/09/11/how-to-install-clojure-on-windows/>
 - <https://leiningen.org/#install>
- Libro:
 - Halloway, S., & Bedra, A. (2012). *Programming Clojure* (2nd ed.). Pragmatic Bookshelf.

Paralelismo funcional: se estudiará de otro libro



Actividades propuestas para cada día

- Day 1: Programming Without Mutable State
 - [Día 1: Programación sin estados mutables]
- Day 2: Functional Parallelism
 - [Día 2: Paralelismo funcional]
- Day 3: Functional Concurrency
 - [Día 3: Concurrencia funcional]



Conclusión del capítulo



• Fortalezas

- El principal beneficio de la programación funcional es la confianza, la seguridad de que su programa hace lo que usted cree que hace.
- Una vez que se ha llegado a pensar funcionalmente, los programas funcionales tienden a ser más simples, más fáciles de razonar y más fáciles de probar que sus equivalentes imperativos.
- Dado que el código funcional elimina el estado mutable, la mayoría de los errores de concurrencia que pueden aparecer en los programas tradicionales basados en hilos y cerrojos son imposibles.



• Debilidades

- Mucha gente espera que el código funcional sea menos eficiente que su equivalente imperativo.

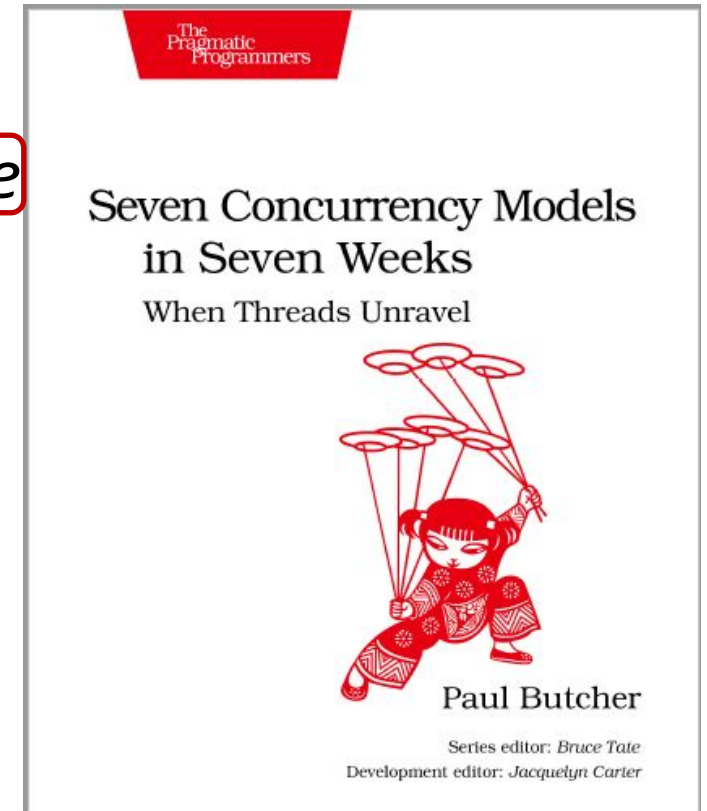
Seven Concurrency Models

- *Introduction*

1. *Threads and locks*
2. *Functional programming*
3. *The Clojure Way—separating identity and state*
4. *Actors*
5. *Communicating Sequential Processes*
6. *Data parallelism*
7. *The Lambda Architecture*
8. *Wrapping Up*

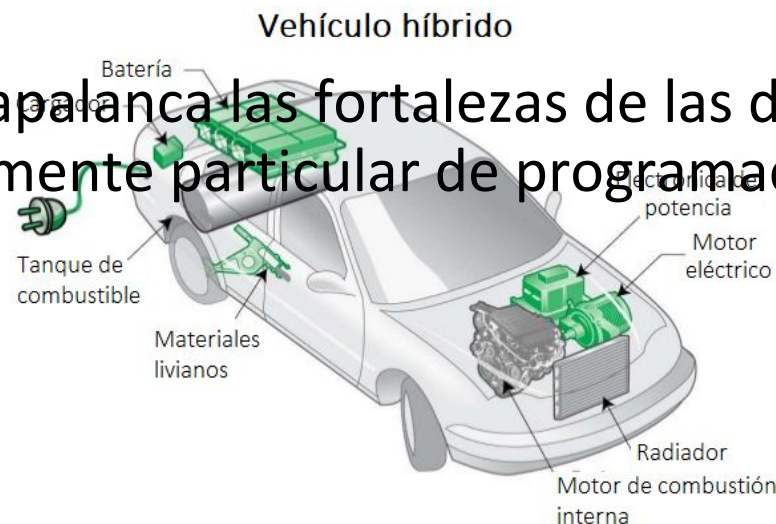
Dando una mirada
rápida a este libro

2014



La “manera” Clojure—separando identidad y estado

- Un vehículo de pasajeros híbrido moderno combina las fortalezas de un motor de combustión interna con aquellas de un motor eléctrico.
 - Dependiente del contexto este algunas veces utiliza solamente potencia eléctrica otras veces solamente gasolina y en otras ocasiones las dos de manera simultánea.
- Clojure proporciona un híbrido similar entre programación funcional y estados mutables.
 - La “manera” Clojure, apalanca las fortalezas de las dos técnicas para proveer un enfoque poderosamente particular de programación concurrente.



Lo mejor de los dos mundos

- Aunque la programación funcional funciona increíblemente bien para algunos problemas, algunos tienen la modificación del estado como elemento fundamental de la solución.
- Aunque puede ser posible crear una solución funcional para tales problemas, es más fácil pensar en ellos de una manera más tradicional.
- En este capítulo iremos más allá del subconjunto funcional puro de Clojure que vimos anteriormente y veremos cómo nos ayuda a crear soluciones concurrentes a tales problemas.

Actividades propuestas para cada día

- Day 1: Atoms and Persistent Data Structures
 - [Día 1: Átomos y estructuras de datos persistentes]
- Day 2: Agents and Software Transactional Memory
 - [Día 2: Agentes y memoria transaccional de software]
- Day 3: In Depth
 - [Día 3: A fondo]



Conclusión del capítulo



• Fortalezas

- Clojure le permite resolver problemas funcionalmente cuando ese es el enfoque natural, pero salirse de la programación funcional pura cuando no sea apropiado.
- Las estructuras de datos persistentes de Clojure permiten que las variables mutables mantengan separadas la identidad y el estado.



• Debilidades

- NO ofrece soporte para programación distribuida (geográficamente o de otro modo).
- No tiene soporte directo para tolerancia a fallos.

¡En este curso no se estudia La "manera" Clojure

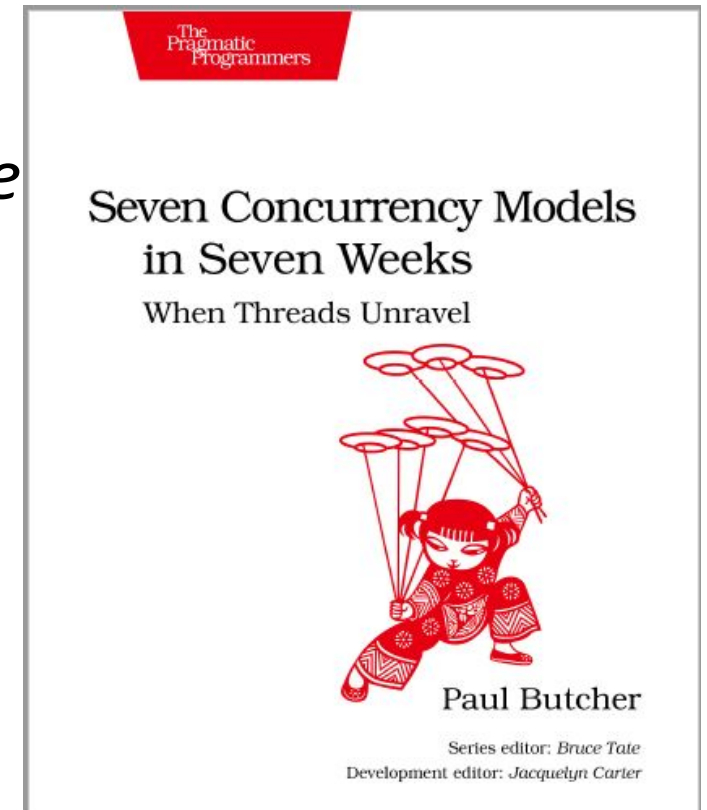
Seven Concurrency Models

- *Introduction*

1. *Threads and locks*
2. *Functional programming*
3. *The Clojure Way—separating identity and state*
4. *Actors*
5. *Communicating Sequential Processes*
6. *Data parallelism*
7. *The Lambda Architecture*
8. *Wrapping Up*

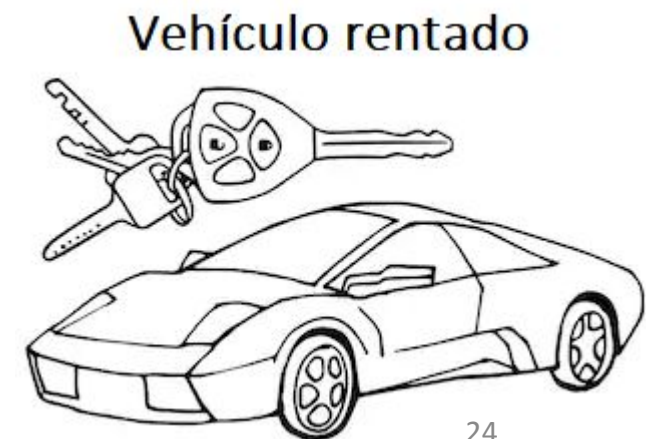
Dando una mirada
rápida a este libro

2014



Actores

- Un actor es como un vehículo rentado, fácil y rápido de conseguir cuando se quiera y si este falla, uno no se preocupa por arreglarlo, solo llama la compañía que se lo rentó y otro vehículo será entregado.
- El modelo de actor es un modelo de programación concurrente de propósito general con una aplicabilidad particularmente amplia, su objetivo es tanto las arquitecturas de memoria compartida como las de memoria distribuida, Facilita la distribución geográfica y proporciona un soporte especialmente fuerte a tolerancia a fallas y resiliencia.



Más orientado a objetos que objetos

- La programación funcional evita los problemas asociados con el estado mutable compartido evitando el estado mutable. La programación de actores, por el contrario, conserva el estado mutable pero evita compartirlo.
- Un actor es como un objeto en un programa orientado a objetos (OO): encapsula el estado y se comunica con otros actores mediante el intercambio de mensajes. La diferencia es que los actores se ejecutan simultáneamente entre sí y, a diferencia del paso de mensajes al estilo OO (que en realidad no es más que invocar un método), los actores se comunican realmente enviándose mensajes entre sí.

Más orientado a objetos que objetos

- Aunque el modelo de actor es un enfoque general de la concurrencia que puede utilizarse con casi cualquier lenguaje, se asocia más con el lenguaje Erlang.
- Vamos a tratar el tema de los actores en Elixir, un lenguaje que se ejecuta en la máquina virtual de Erlang (BEAM).
- Al igual que Clojure (y Erlang), Elixir es un lenguaje funcional impuro, dinámicamente tipado.

Actividades propuestas para cada día

- Day 1: Messages and Mailboxes
 - [Día 1: Mensajes y buzzone]
- Day 2: Error Handling and Resilience
 - [Día 2: Manejo de errores y resiliencia]
- Day 3: Distribution
 - [Día 3: Distribución]

Conclusión del capítulo

- Podemos pensar en los actores como la extensión lógica de la programación orientada a objetos al mundo concurrente. De hecho, se puede pensar en los actores como algo más orientado a objetos que los objetos, con un paso de mensajes y un encapsulamiento más estrictos.

• Fortalezas

- Los actores no comparten estado y, aunque se ejecutan de forma concurrente entre sí, dentro de un mismo actor todo es secuencial. Esto significa que sólo debemos preocuparnos por la concurrencia cuando consideramos los flujos de mensajes entre actores.
- La tolerancia a fallos se incorpora a los programas de actores desde el principio. Esto no sólo permite que los programas sean más resistentes, sino también que el código sea más sencillo y claro (gracias a la filosofía de "dejar que falle" ["let it crash"]).
- La compatibilidad de los actores con arquitecturas de memoria compartida y distribuida aporta una serie de ventajas significativas:
 - permite escalar un programa actor para resolver problemas de casi cualquier tamaño.
 - nos permite abordar problemas en los que la distribución geográfica es una consideración intrínseca.
 - la distribución es un factor clave para los sistemas resilientes y tolerantes a fallos.

Conclusión del capítulo



• Debilidades

- Los actores siguen siendo susceptibles a problemas como el bloqueo, además de algunos modos de fallo exclusivos de los actores (como el desbordamiento del buzón de un actor).
- Los actores NO proporcionan soporte directo para el paralelismo.
- Y debido a que los actores no comparten estado y sólo pueden comunicarse a través del paso de mensajes, no son una opción adecuada si necesitas un paralelismo de grano fino.
- El modelo de actor no es nuevo, se describió por primera vez en la década de 1970, sobre todo por Carl Hewitt.
- Si está interesado en aprender más sobre Akka, consulte el capítulo extra en línea, que describe la programación de actores en Scala.

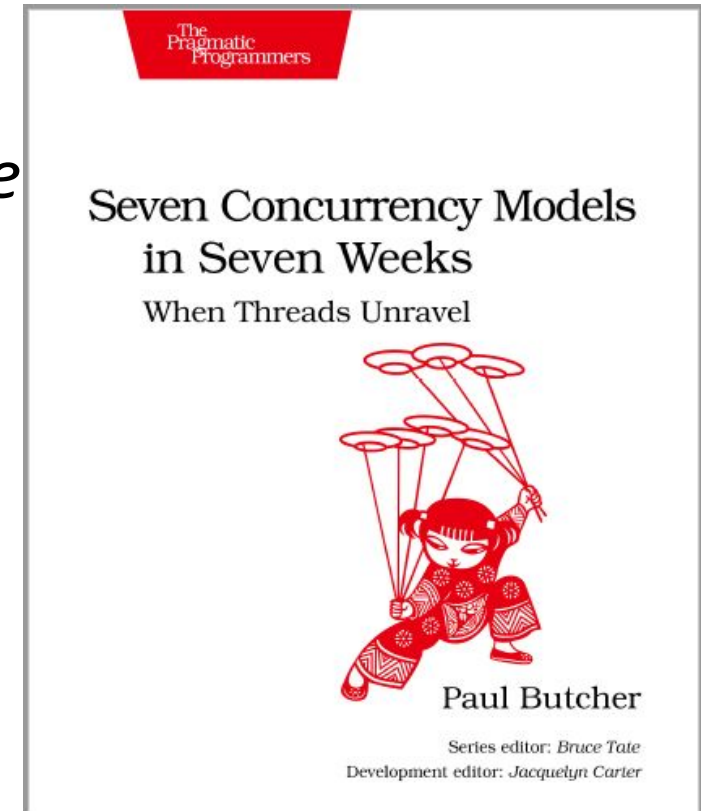
Seven Concurrency Models

- *Introduction*

1. *Threads and locks*
2. *Functional programming*
3. *The Clojure Way—separating identity and state*
4. *Actors*
5. *Communicating Sequential Processes*
6. *Data parallelism*
7. *The Lambda Architecture*
8. *Wrapping Up*

Dando una mirada
rápida a este libro

2014



Communicating Sequential Processes

- Donde podamos ir y qué tan rápido podamos ir es definido por la red de carreteras, no sólo por el vehículo.
- De manera parecida, las características y capacidades de un sistema de paso mensajes no son solo definidas por el código que se utiliza para intercambiar los dichos mensajes o por el contenido de los mismos, sino por el transporte sobre el cual ellos se mueven



La comunicación lo es todo

- Como vimos en el capítulo anterior, un programa de actores consiste en entidades independientes que se ejecutan simultáneamente (llamadas actores o procesos en Elixir) y que se comunican enviándose mensajes entre sí. Cada actor tiene un buzón que almacena los mensajes hasta que se gestionan.
- Un programa que utilice el modelo de procesos secuenciales que se comunican consta, de forma similar, de entidades independientes que se ejecutan simultáneamente y que se comunican enviándose mensajes entre sí. La diferencia radica en el énfasis: en lugar de centrarse en las entidades que envían los mensajes, CSP se centra en los canales a través de los cuales se envían. Los canales son de primera clase: en lugar de que cada proceso esté estrechamente vinculado a un único buzón de correo, los canales pueden crearse, escribirse, leerse y transmitirse entre procesos de forma independiente.

La comunicación lo es todo

- Al igual que la programación funcional y los actores, CSP es una vieja idea que está experimentando un renacimiento. La reciente popularidad de CSP se debe en gran parte al lenguaje Go.
- Vamos a cubrir CSP examinando la librería `core.async`, que lleva el modelo de concurrencia de Go a Clojure.

Actividades propuestas para cada día

- Day 1: Channels and Go Blocks
 - [Día 1: Canales y bloques del lenguaje Go]
- Day 2: Multiple Channels and IO
 - [Día 2: Canales múltiples y operaciones de E/S]
- Day 3: Client-Side CSP
 - [Día 3: CSP del lado del cliente]



Conclusión del capítulo



• Fortalezas

- La principal fortaleza de CSP en comparación con los actores es la flexibilidad.
- En un programa con actores, el medio de comunicación está estrechamente vinculado a la unidad de ejecución: cada actor tiene precisamente un buzón de correo. En un programa CSP, por el contrario, los canales son componentes de primera clase y pueden crearse, escribirse, leerse y pasarse entre tareas de forma independiente.

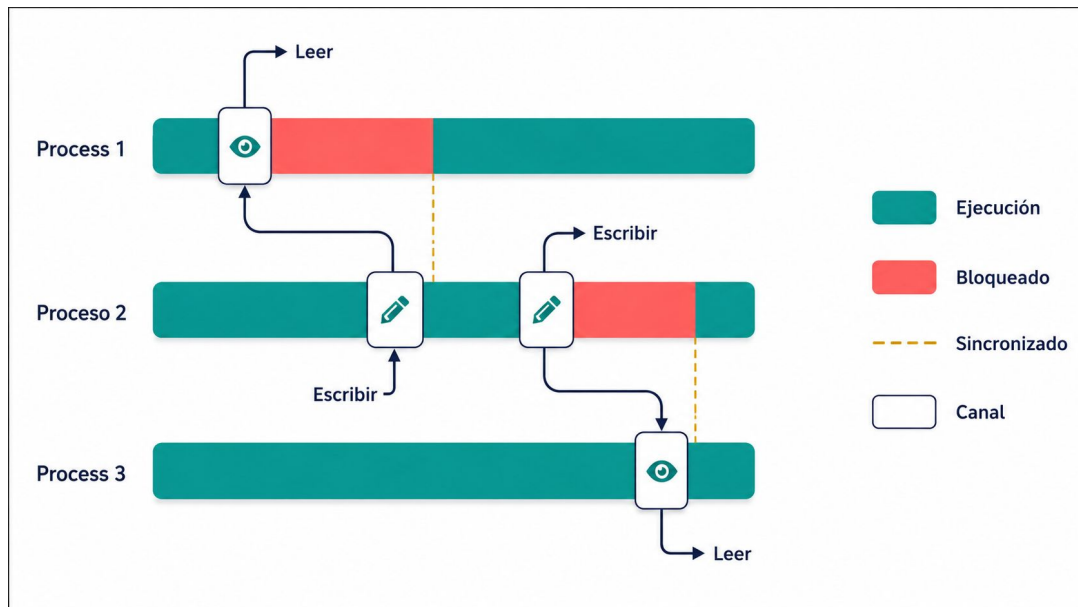


• Debilidades

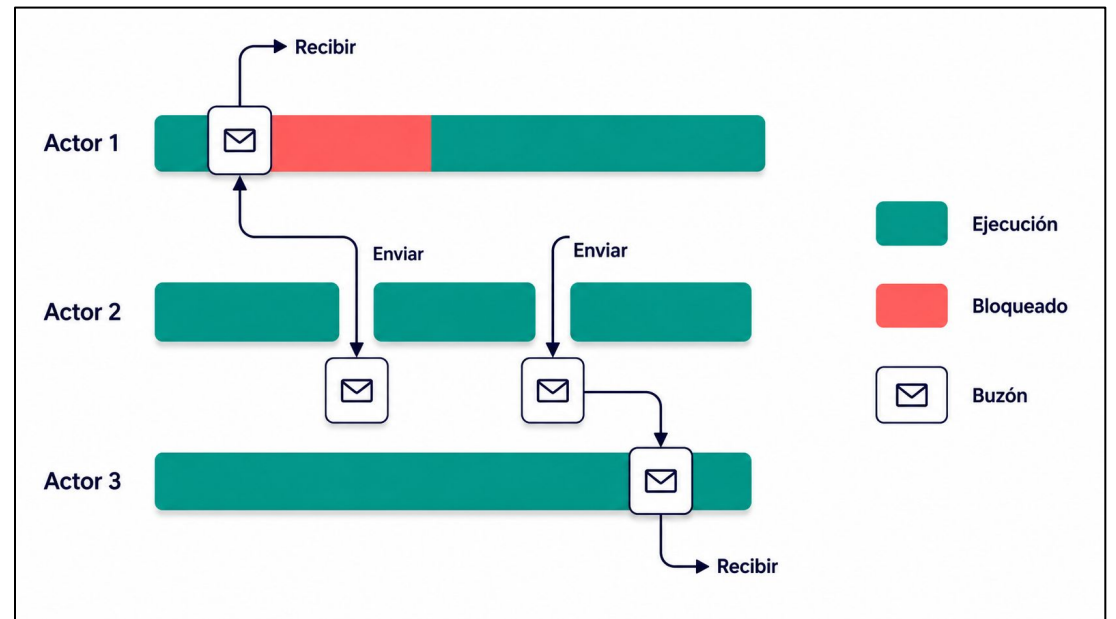
- Aunque no hay nada que impida que los lenguajes basados en CSP soporten la distribución y la tolerancia a fallos, históricamente ninguna de ellas ha tenido el mismo nivel de atención y soporte que los lenguajes basados en actores.
- Al igual que ocurre con los hilos, los cerrojos y los actores, los programas CSP son susceptibles de bloquearse y no tienen soporte directo para el paralelismo.
- Al igual que los actores, CSP existe desde los años setenta, cuando fue introducida por Tony Hoare. Los dos modelos han evolucionado juntos, aprendiendo el uno del otro a lo largo de los años.

Comparando el modelo de Actores y CSP

Communicating Sequential Processes (CSP)

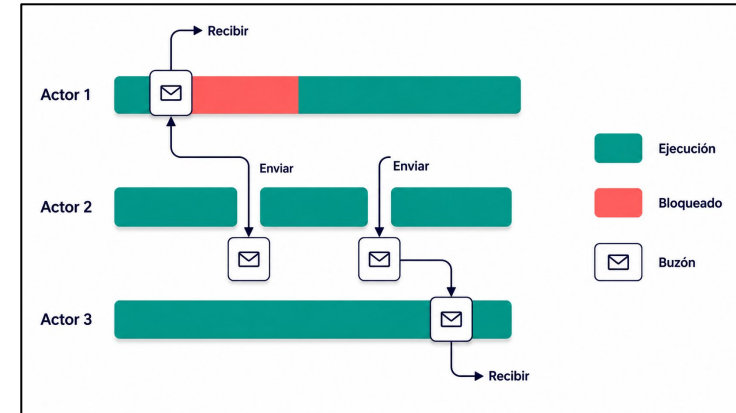
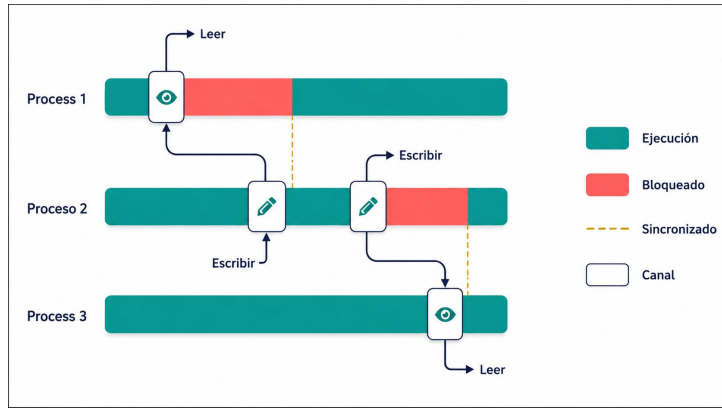


Actor model



<https://dev.to/karanpratapsingh/csp-vs-actor-model-for-concurrency-1cpg>

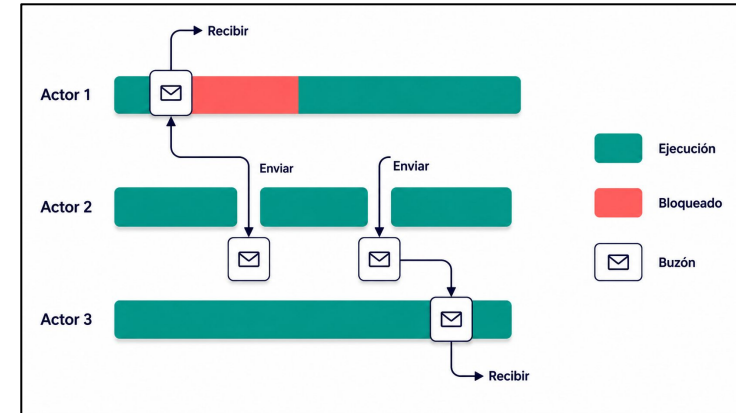
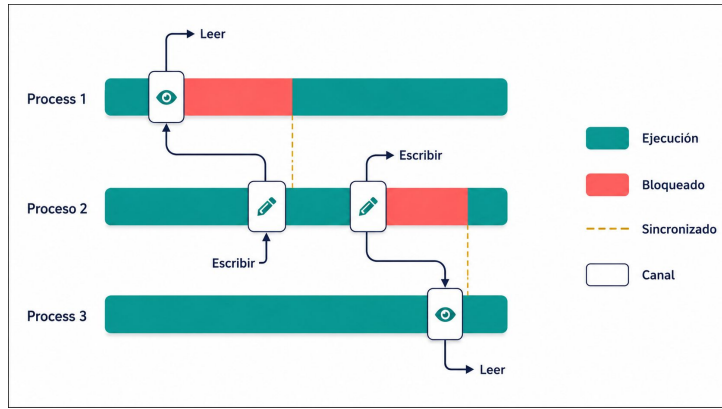
Comparando el modelo de Actores y CSP



Ambos modelos buscan **coordinar actividades concurrentes sin depender directamente de memoria compartida**, pero difieren en la forma de comunicación.

- En **CSP (Communicating Sequential Processes)**, los procesos se comunican mediante **canales**, y las operaciones de lectura y escritura pueden actuar como puntos explícitos de **sincronización**, por lo que un proceso puede quedar bloqueado esperando que otro participe en la comunicación.

Comparando el modelo de Actores y CSP



- En el **modelo de actores**, en cambio, cada actor dispone de un **buzón (mailbox)** y se comunica mediante el **envío asíncrono de mensajes**: el emisor deposita el mensaje y puede continuar su ejecución sin esperar a que el receptor lo procese.

Así, mientras CSP enfatiza la **comunicación sincronizada mediante canales**, el modelo de actores enfatiza la **comunicación asíncrona mediante mensajes y buzones**.

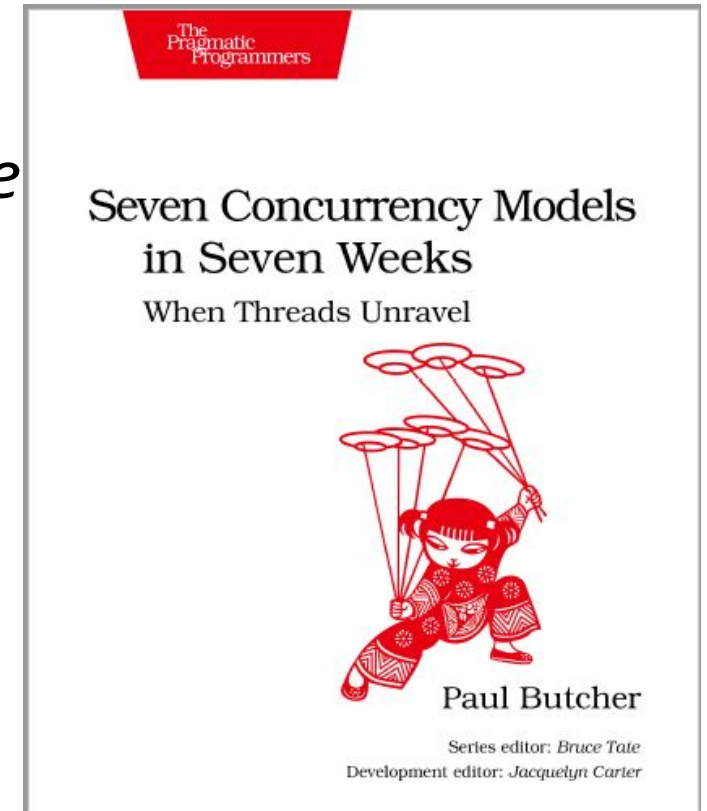
Seven Concurrency Models

- *Introduction*

1. *Threads and locks*
2. *Functional programming*
3. *The Clojure Way—separating identity and state*
4. *Actors*
5. *Communicating Sequential Processes*
6. *Data parallelism*
7. *The Lambda Architecture*
8. *Wrapping Up*

Dando una mirada
rápida a este libro

2014



Paralelismo de datos (Data parallelism)

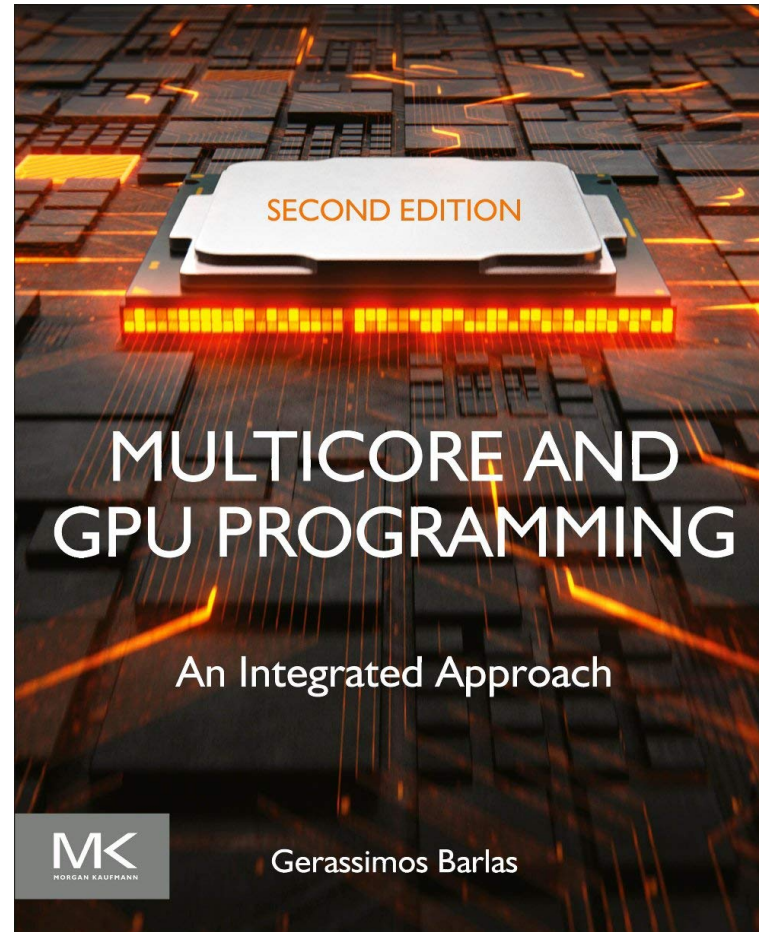
- El paralelismo de datos es como una autopista de varios carriles, aun cuando cada vehículo está viajando una velocidad relativamente modesta, el número de vehículos que pasan por un punto particular es grande debido a la cantidad de vehículos que pueden usar la vía.
- Los modelos que fueron discutidos hasta aquí son aplicables a una variedad amplia de problemas. En contraste, la programación en paralelismo de datos es relevante únicamente a un rango más selecto, más específico. Como su nombre sugiere, esta es una técnica de programación paralela, no una técnica de diseño concurrente (recordemos que la concurrencia y paralelismo no es lo mismo)



La supercomputadora oculta en su portátil

- En este capítulo vamos a ver cómo aprovechar el superordenador oculto en tu portátil: la unidad de procesamiento gráfico o GPU. Una GPU moderna es un potente procesador paralelo de datos, capaz de eclipsar a la CPU cuando se utiliza para hacer cálculos numéricos, una práctica que suele denominarse cálculo de propósito general en la GPU o programación GPGPU [*general-purpose computing on the GPU*].
- A lo largo de los años, han surgido una serie de tecnologías que permiten abstraerse de los detalles de implementación de la GPU. Utilizaremos el Open Computing Language, u OpenCL, para escribir código GPGPU.

Paralelismo de datos : se estudiará de otro libro



Actividades propuestas para cada día

- Day 1: GPGPU Programming
 - [Día 1: Programación GPGPU]
- Day 2: Multiple Dimensions and Work-Groups
 - [Día 2: Dimensiones múltiples y grupos de trabajo]
- Day 3: OpenCL and OpenGL—Keeping It on the GPU
 - [OpenCL y OpenGL - Mantener la GPU en funcionamiento]



Conclusión del capítulo



• Fortalezas

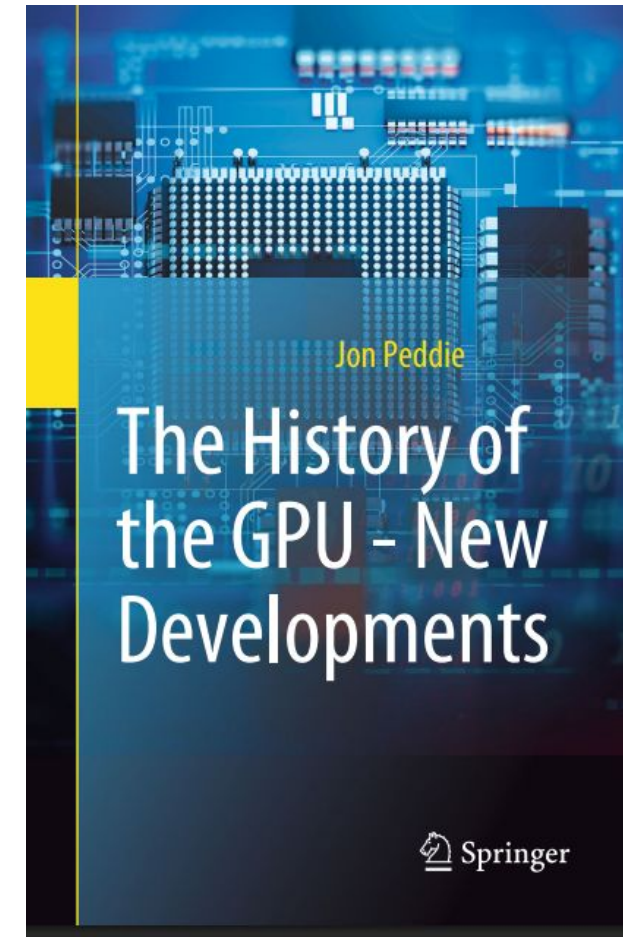
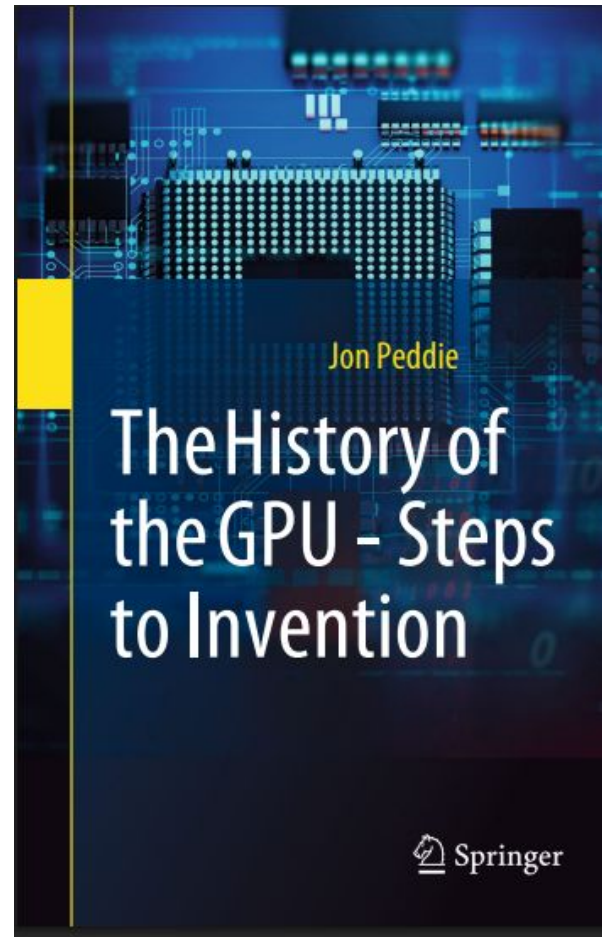
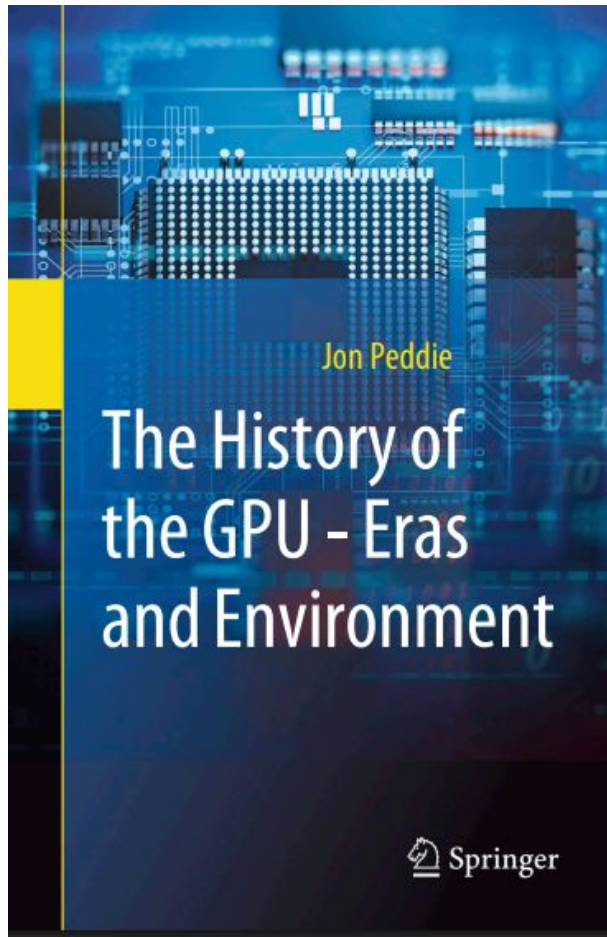
- El paralelismo de datos es ideal siempre que te enfrentes a un problema en el que haya que procesar grandes cantidades de datos numéricos.
- Las GPU no sólo son potentes procesadores de paralelismo de datos; también son extremadamente eficientes en su consumo de energía.



• Debilidades

- En concreto, aunque es posible utilizar estas técnicas para crear soluciones a problemas no numéricos (procesamiento del lenguaje natural, por ejemplo), hacerlo no es sencillo: el conjunto de herramientas actual está muy centrado en el cálculo numérico. [esto ya no es tan cierto... Recordemos deep learning]

Otros libros...



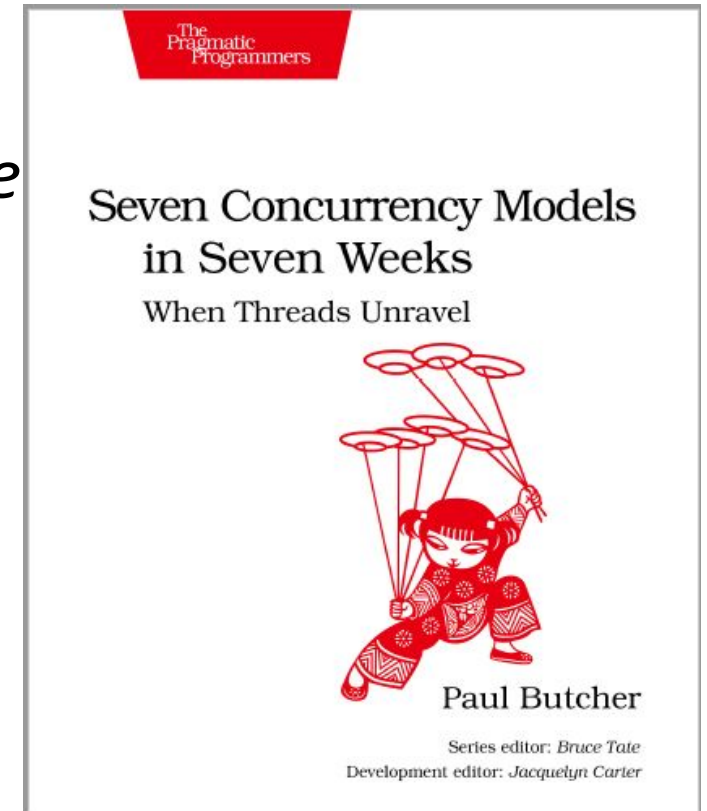
Seven Concurrency Models

- *Introduction*

1. *Threads and locks*
2. *Functional programming*
3. *The Clojure Way—separating identity and state*
4. *Actors*
5. *Communicating Sequential Processes*
6. *Data parallelism*
7. *The Lambda Architecture*
8. *Wrapping Up*

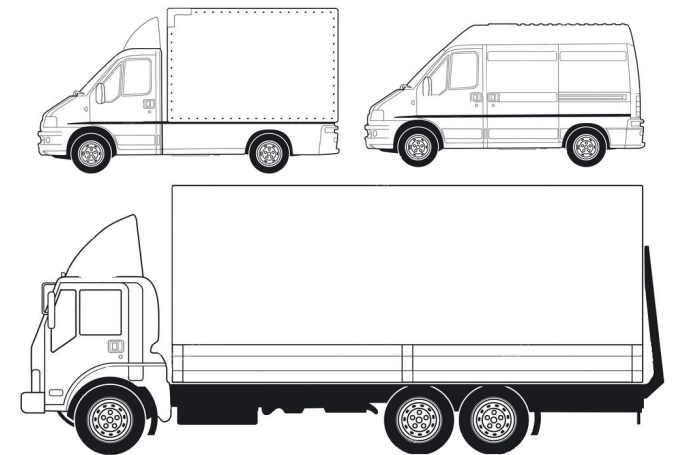
Dando una mirada
rápida a este libro

2014



La Arquitectura Lambda [The Lambda Architecture]

- Si necesitamos enviar mucha carga de un lado del continente a otro, nada puede superar un camión de 18 ruedas. Sin embargo no sería la selección correcta para enviar un único paquete, por esta razón una compañía de envíos integrales también tiene una flotilla de camionetas que realizan recolecciones y entregas de ese tipo.
- La arquitectura lambda combina grandes fortalezas de procesamiento por lotes de MapReduce con la capacidad de respuesta en tiempo real de procesamiento de streams para permitirnos crear soluciones escalables con capacidad de respuesta y tolerante a fallas para Big Data.



El paralelismo permite Big Data

- La llegada de los macrodatos [Big Data] ha supuesto un cambio radical en el procesamiento de datos en los últimos años.
 - El Big Data se diferencia del procesamiento de datos tradicional por su uso del paralelismo: solo mediante la utilización de múltiples recursos informáticos podemos contemplar el procesamiento de terabytes de datos.
 - La arquitectura Lambda es un enfoque particular de Big Data popularizado por Nathan Marz, derivado de su etapa en BackType y posteriormente en Twitter.
- Al igual que el tema de la semana pasada, la programación GPGPU, la Arquitectura Lambda aprovecha el paralelismo de datos.
 - La diferencia es que lo hace a gran escala, distribuyendo tanto los datos como el cálculo en clústeres de decenas o cientos de máquinas.
 - Esto no sólo proporciona la potencia suficiente para hacer abordables problemas antes intratables, sino que también nos permite crear sistemas tolerantes a fallos tanto de hardware como humanos.

¿Qué es la arquitectura Lambda?

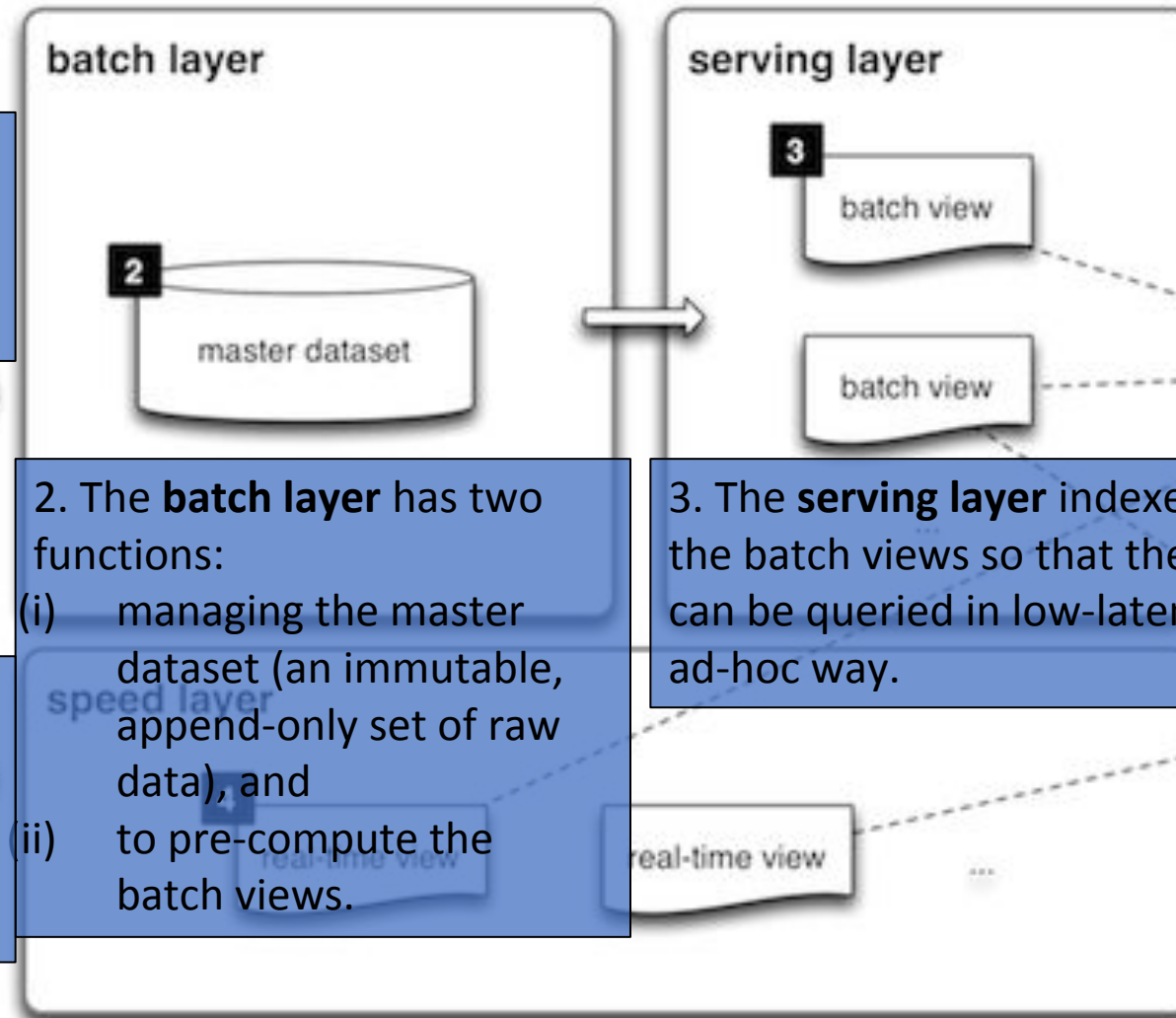
- Nathan Marz ideó el término Arquitectura Lambda [Lambda Architecture] (LA) para una arquitectura de procesamiento de datos genérica, escalable y tolerante a fallos, basándose en su experiencia trabajando en sistemas de procesamiento de datos distribuidos en Backtype y Twitter.
- La Lambda Architecture (LA) pretende satisfacer las necesidades de un sistema robusto y tolerante a fallos, tanto de hardware como humanos, capaz de servir a una amplia gama de cargas de trabajo y casos de uso, y en el que se requieran lecturas y actualizaciones de baja latencia. El sistema resultante debe ser linealmente escalable, y debe escalar hacia fuera en lugar de hacia arriba.

¿Qué es la arquitectura Lambda?

The *batch layer* uses **batch-oriented technologies** like MapReduce to precompute **batch views** from historical data.

This is effective, but **latency is high**, so we add a *speed layer* that uses **low-latency techniques** like stream processing to create **real-time views** from new data as it arrives.

The two types of views are then combined to create query results.



We're going to concentrate on its two primary building blocks, the batch layer and the speed layer

The Lambda Architecture is by far the most complicated subject we're going to cover in this book. It builds upon several underlying technologies, the most important of which is MapReduce.

Actividades propuestas para cada día

- Day 1: MapReduce
 - Día 1: MapReduce
- Day 2: The Batch Layer
 - Día 2: Capa Batch
- Day 3: The Speed Layer
 - Día 3: Capa Speed



Conclusión del capítulo

- La arquitectura Lambda reúne muchos conceptos que ya hemos tratado en otras ocasiones:
 - La idea de que los datos en bruto [raw data] son “eternamente verdadero”s debería recordarle a la separación de identidad y estado de Clojure.
 - El enfoque de Hadoop de paralelizar un problema dividiéndolo en un mapa sobre una estructura de datos seguido de una reducción debería recordarle mucho de lo que vimos cuando estudiamos la programación funcional paralela.
 - <https://hadoop.apache.org/>
 - Al igual que los actores, la arquitectura Lambda distribuye el procesamiento en un clúster para mejorar el rendimiento y proporcionar tolerancia a fallos ante fallos de hardware.
 - <http://lambda-architecture.net/>
 - Los flujos de tuplas de Storm tienen mucho en común con el paso de mensajes que vimos tanto en actores como en CSP.
 - <http://storm.apache.org/releases/current/Understanding-the-parallelism-of-a-Storm-topology.html>

Conclusión del capítulo



• Fortalezas

- The Lambda Architecture is all about handling huge quantities of data—problems where traditional data-processing architectures are struggling to cope. It's particularly well suited to reporting and analytics—the kinds of problems that might have been addressed with a data warehouse in the past.

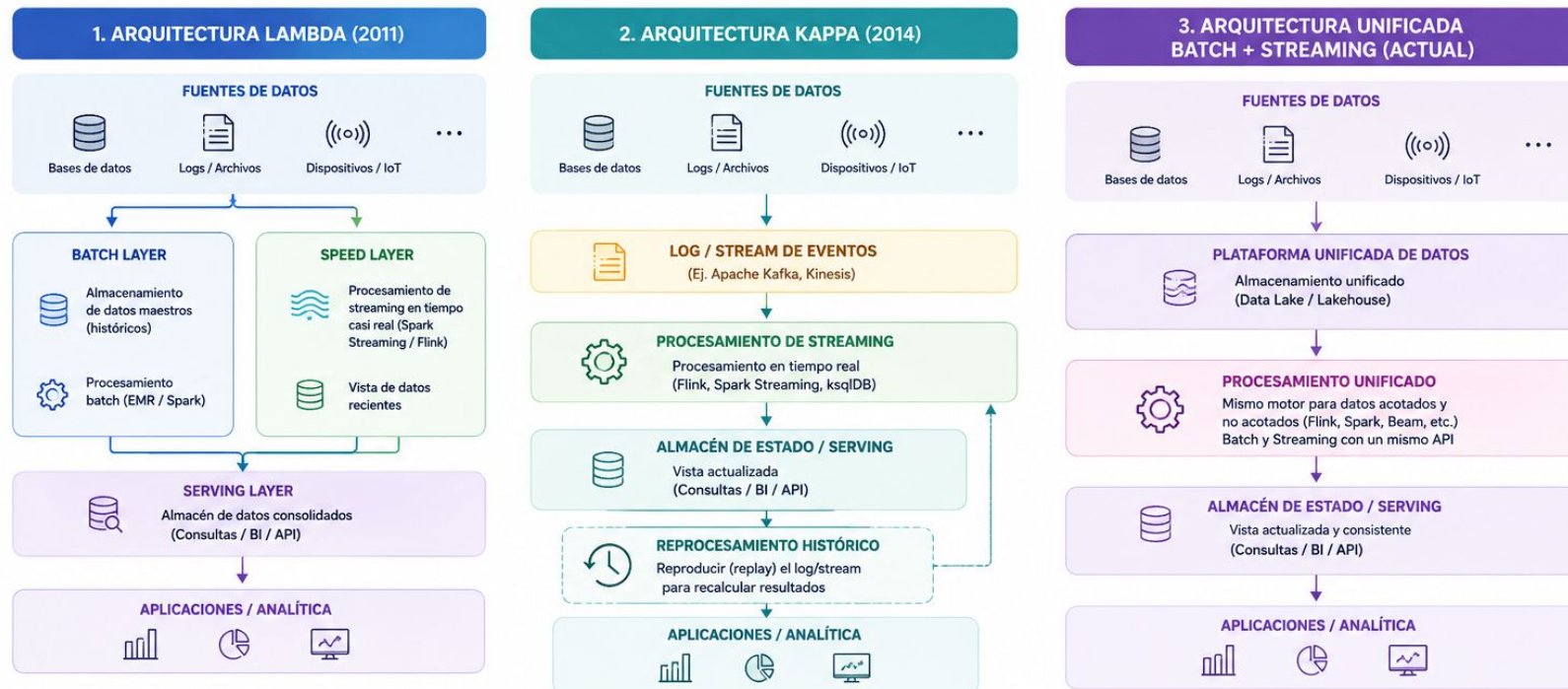


• Debilidades

- The Lambda Architecture's great strength—its focus on huge quantities of data—is also its weakness. Unless your data is measured in tens of gigabytes or more, its overhead (both computational and intellectual) is unlikely to be worth the benefit.

Evolución de Lambda, Kappa y Arquitectura unificada Batch + Streaming

EVOLUCIÓN DE ARQUITECTURAS DE PROCESAMIENTO DE DATOS De Lambda (2011) a Kappa (2014) y hacia el procesamiento unificado de Batch y Streaming

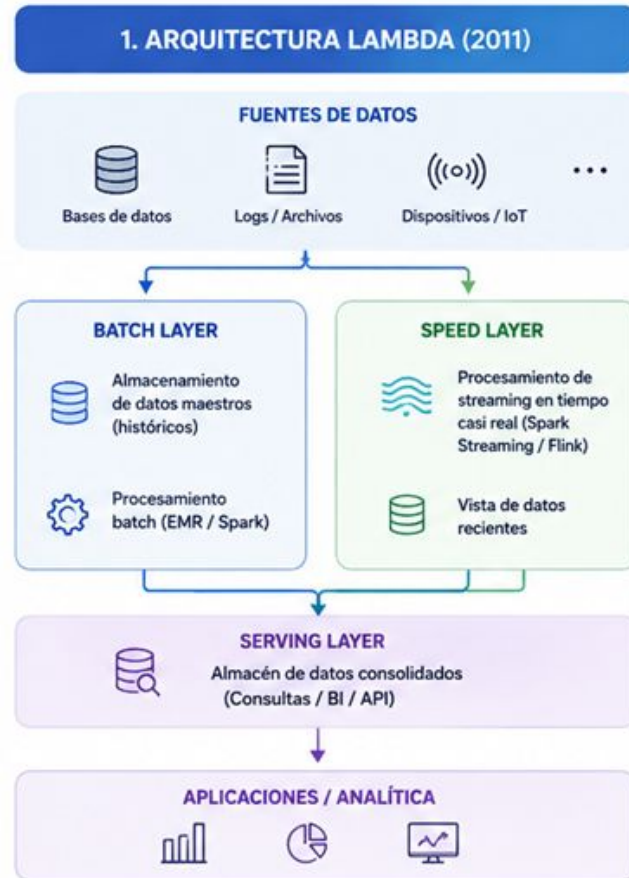


ASPECTO	LAMBDA	KAPPA	UNIFICADA (ACTUAL)
Modelo de datos	Dos caminos: Batch + Streaming	Un solo flujo de eventos (stream)	Unificado: batch y streaming en el mismo motor
Complejidad	Alta (dos bases de código y capas)	Media (una base de código)	Menor (una base de código y plataforma)
Datos históricos	Procesados en batch periódicamente	Replay del log/stream para reprocesar	Reprocesamiento eficiente con el mismo motor
Costo operativo	Más alto (duplicación de recursos)	Menor	Optimizado (uso eficiente de recursos)
Casos de uso	Analítica + tiempo real con baja latencia	Tiempo real, eventos, IoT	Analítica en tiempo real, ML, lakehouse, event-driven

TECNOLOGÍAS COMUNES

- Almacenamiento: S3, HDFS, Delta Lake, Iceberg, Hudi
- Streaming / Log: Kafka, Kinesis, Pulsar
- Procesamiento: Spark, Flink, Beam, EMR, Databricks
- Serving: Redshift, Druid, Elasticsearch, ClickHouse

Evolución de Lambda, Kappa y Arquitectura unificada Batch + Streaming



- La evolución de Lambda → Kappa → Arquitectura Unificada Batch + Streaming responde principalmente a la búsqueda de menor complejidad, menor duplicación y procesamiento más flexible de los datos.

La Arquitectura Lambda surgió para combinar el procesamiento histórico (batch) con resultados de baja latencia (streaming), pero exige mantener dos rutas de procesamiento y, frecuentemente, duplicar lógica.

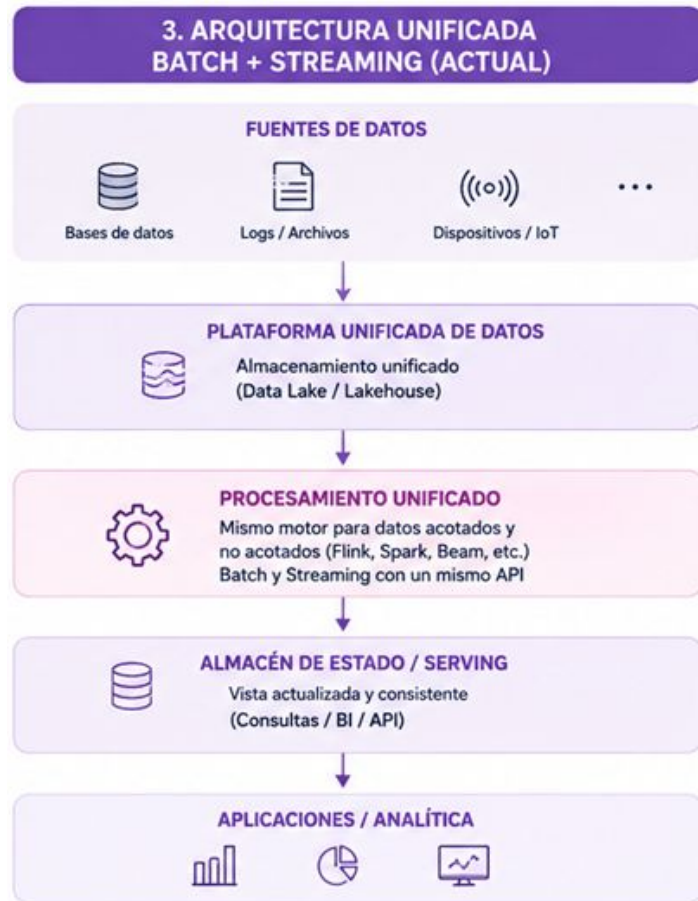
Evolución de Lambda, Kappa y Arquitectura unificada Batch + Streaming



- La evolución de Lambda → Kappa → Arquitectura Unificada Batch + Streaming responde principalmente a la búsqueda de menor complejidad, menor duplicación y procesamiento más flexible de los datos.

Kappa simplificó este enfoque al tratar los datos como un flujo continuo de eventos y utilizar la reproducción (replay) del flujo para recalcular información histórica, eliminando la ruta batch independiente.

Evolución de Lambda, Kappa y Arquitectura unificada Batch + Streaming



- La evolución de Lambda → Kappa → Arquitectura Unificada Batch + Streaming responde principalmente a la búsqueda de menor complejidad, menor duplicación y procesamiento más flexible de los datos.

Las arquitecturas unificadas actuales aprovechan motores capaces de procesar con un mismo modelo datos acotados (batch) y no acotados (streaming), conservando las ventajas del tiempo real pero facilitando también el reprocesamiento histórico

Evolución de Lambda, Kappa y Arquitectura unificada Batch + Streaming

EVOLUCIÓN DE ARQUITECTURAS DE PROCESAMIENTO DE DATOS
De Lambda (2011) a Kappa (2014) y hacia el procesamiento unificado de Batch + Streaming



La evolución busca pasar de dos caminos separados, a un solo flujo y, finalmente, a un único modelo de procesamiento para ambos tipos de datos.

¡En este curso no se estudia La Arquitectura Lambda

ASPECTO	LAMBDA	KAPPA	ACTUAL
Modelo de datos	Dos caminos: Batch + Streaming	Un solo flujo de eventos (stream)	Un solo flujo de eventos (stream)
Complejidad	Alta (dos bases de código y capas)	Media (una base de código)	Baja (una base de código y plataforma)
Datos históricos	Procesados en batch periódicamente	Replay del log/stream para reprocessar	Replay del log/stream para reprocessar
Costo operativo	Más alto (duplicación de recursos)	Menor	Optimizado (uso eficiente de recursos)
Casos de uso	Analítica + tiempo real con baja latencia	Tiempo real, eventos, IoT	Analítica en tiempo real, ML, lakehouse, event-driven

TECNOLOGÍAS COMUNES

- Almacenamiento: S3, HDFS, Delta Lake, Iceberg, Hudi
- Streaming / Log: Kafka, Kinesis, Pulsar
- Procesamiento: Spark, Flink, Beam, EMR, Databricks
- Serving: Redshift, Druid, Elasticsearch, ClickHouse

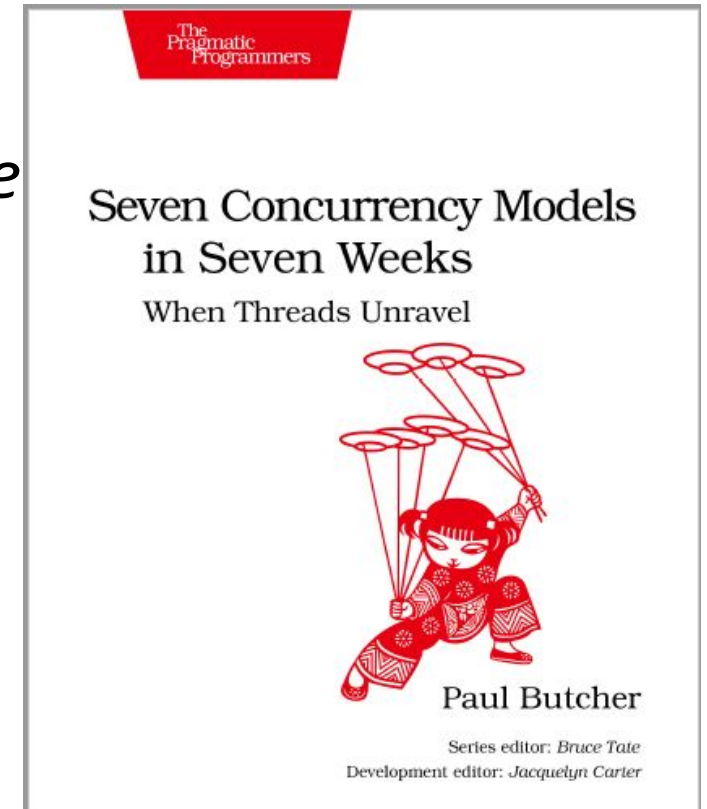
Seven Concurrency Models

- *Introduction*

1. *Threads and locks*
2. *Functional programming*
3. *The Clojure Way—separating identity and state*
4. *Actors*
5. *Communicating Sequential Processes*
6. *Data parallelism*
7. *The Lambda Architecture*
8. *Wrapping Up*

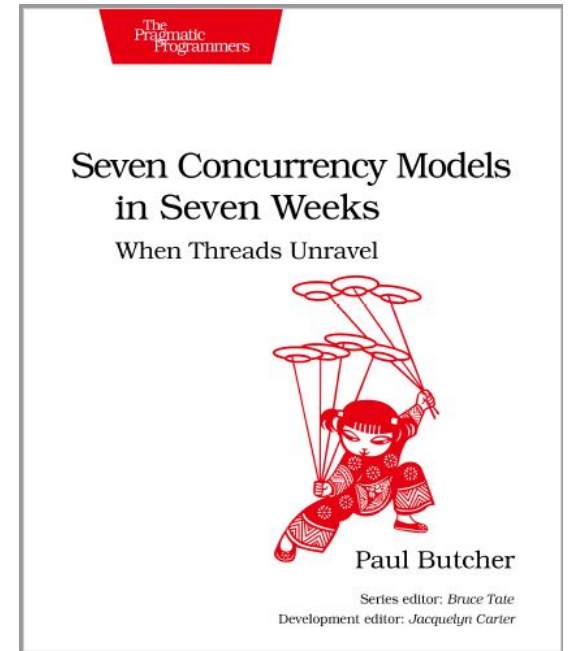
Dando una mirada
rápida a este libro

2014



Resumen del libro

- El futuro es inmutable
- El futuro está distribuido
- Caminos no tomados en este libro:
 - Fork/Join y robo de trabajo [Fork/Join and Work-Stealing]
 - Flujo de datos [Dataflow]
 - Programación reactiva [Reactive Programming]
 - Programación reactiva funcional [Functional Reactive Programming]
 - Computación Grid [Grid Computing]
 - Espacios de tuplas [Tuple Spaces]



En el curso **SÍ** se estudia:

1. Fork/Join
2. Algoritmo de Work Stealing
3. Paralelismo con flujo de datos



Gracias